

Name:Niranjan Bokhare

PRN:202501050060

Batch:CH3

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667a7e0428220463f497a3c

CODETANTRA Home Learn Anywhere

202501050060@mitaoe.ac.in Support Logout

4.1.1. Pandas - series creation and manipulation 04:10

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.
Hint: You can group the Series based on whether each number is even or odd.
Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 s = pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = s.groupby(s % 2 == 0).mean()
10
11 # Display the mean of even and odd numbers with labels
12 grouped.index = ['Even' if is_even else 'Odd' for is_even in
13 grouped.index]
14 print("Mean of even and odd numbers:")
15 print(grouped)

Average time 0.024 s
Maximum time 0.055 s
3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 55 ms
Expected output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd5.0
Even6.0
dtype: float64
Actual output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd5.0
Even6.0
dtype: float64

Terminal Test cases

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667aa30728220463f649bb62

CODETANTRA Home Learn Anywhere

202501050060@mitaoe.ac.in Support Logout

4.1.2. Dictionary to dataframe 04:25

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.
Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.

datafram...

1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5 'Name': ['Alice', 'Bob', 'Charlie'],
6 'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 name = input("Enter name: ")
17 age = int(input("Enter age: "))
18 new_row = {}
19
20 # Display the DataFrame after adding a new row
21 print("After adding a row:\n",df)
22
23 mod_index = int(input("Enter row index to modify:"))
24 new_age_val = int(input("Enter new age:"))
25 df.at[mod_index, 'Age']
26 # Display the DataFrame after modifying a row
27 print("After modifying a row:")
28 print(df)
29

Terminal Test cases

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667cf60f2365ba61af5a2ce7

CODETANTRA Home Learn Anywhere

202501050060@mitaoe.ac.in Support Logout

4.1.3. Student Information 20:31

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).

studentin...

1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep='\\s+', header=None, names=["Name", "Age",
6 "Grade"])

Terminal Test cases

• Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

```
6
7
8 # write your code here..
9 print("First five rows:")
10 print(data.head())
11
12 avg_age = round(data["Age"].mean(), 2)
13 print(f"Average age: {avg_age}")
14
```

Average time
0.145 s
145.00 ms

Maximum time
0.198 s
198.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1
Expected output
studentdata.txt
First five rows:
...Name Age Grade
0 John 25 A
1 Alin 22 B
2 Emma 24 A
Actual output
studentdata.txt
First five rows:
...Name Age Grade
0 John 25 A
1 Alin 22 B
2 Emma 24 A

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 df['Date'] = pd.to_datetime(df['Date'])
10 df['Month'] = df['Date'].dt.strftime("%Y-%m")
11 df['Total Sales'] = df['Quantity'] * df['Price']
12
13 # Find the month with the highest total sales
14 sales_by_month = df.groupby('Month')['Total Sales'].sum()
15 best_month = sales_by_month.idxmax()
```

Average time
0.070 s
69.67 ms

Maximum time
0.152 s
152.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1
Expected output
sales_data.csv
Best month: 2025-01
Total sales: \$1210.00
Actual output
sales_data.csv
Best month: 2025-01
Total sales: \$1210.00

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 product_sales = df.groupby("Product")["Quantity"].sum()
10 # Find the product with the highest total quantity sold
11 best_product = product_sales.idxmax()
12 highest_quantity = product_sales.max()
13
14 # Display the result
15 print(f"Best selling product: {best_product}")
```

Average time
0.057 s
57.00 ms

Maximum time
0.113 s
113.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test case 1	113 ms	Debug	
Expected output	Actual output		
sales_data.csv	sales_data.csv		
Best selling product: Product A	Best selling product: Product A		
Total quantity sold: 23	Total quantity sold: 23		

Terminal

Test cases

Thunderstorm w...
In effect

Search

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67881553a50fe56b8dcf9a6e

CODETANTRA Home Learn Anywhere

202501050060@mitaoe.ac.in Support Logout

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df = pd.read_csv(file_name)
9
10 # write the code..
11 city_sales = df.groupby("City")["Quantity"].sum()
12 best_city=city_sales.idxmax()
13
14
15
```

Average time 0.054 s 54.33 ms Maximum time 0.113 s 113.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1	113 ms	Debug	
Expected output	Actual output		
sales_data.csv	sales_data.csv		
City sold the most products: Los Angeles	City sold the most products: Los Angeles		

Terminal

Test cases

Thunderstorm w...
In effect

Search

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67881553a50fe56b8dcf9a6e

CODETANTRA Home Learn Anywhere

202501050060@mitaoe.ac.in Support Logout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appear in transactions on 2025-01-04.

frequently...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12 date_products = {}
13
14 for date, group in df.groupby('Date'):
```

Average time 0.068 s 68.33 ms Maximum time 0.161 s 161.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1	161 ms	Debug	
Expected output	Actual output		
sales_data.csv	sales_data.csv		
Product A and Product B: 2 times	Product A and Product B: 2 times		
Product A and Product C: 2 times	Product A and Product C: 2 times		

Terminal

Test cases

Course

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6a/67e2b9309025661df3aa574f

202501050060@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.

2. Display the last 5 rows of the dataset.

3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using .info()).

5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().

6. Check for missing values and display the count of missing values for each column.

7. Fill missing values in the 'Age' column with the median age.

8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).

9. Drop the 'Cabin' column due to many missing values.

10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	S	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2	31.01282	7.925	S

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2 3101282,7.925,,S

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6

7

8import pandas as pd

9import numpy as np

10

11# Load the Titanic dataset

12data = pd.read_csv('Titanic-Dataset.csv')

13

14# 1. Display the first 5 rows of the dataset

15print(data.head())

Average time0.486 s

Maximum time0.486 s

1 out of 1 shown test case(s) passed

Test case 1486 ms

Expected output

Actual output

PassengerId - Survived - Pclass - Fare - Cabin - Embarked

0 - NaN - 5 - 7.2500 - NaN - S

1 - 2 - 1 - 71.2833 - C85 - C

2 - 3 - 1 - 7.9250 - NaN - S

TerminalTest cases

Course

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6a/67e378a17028471d94dad0a6

202501050060@mitaoe.ac.inSupportLogout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.

2. Convert the 'Sex' column to numeric values (male: 0, female: 1).

3. One-hot encode the 'Embarked' column, dropping the first category.

4. Get the mean age of passengers.

5. Get the median fare of passengers.

6. Get the number of passengers by class.

7. Get the number of passengers by gender.

8. Get the number of passengers by survival status.

9. Calculate the survival rate of passengers.

10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	S	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2	31.01282	7.925	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6data['FamilySize'] = data['SibSp'] + data['Parch']

7

8

9# 1. Create IsAlone column

10data['FamilySize'] = data['SibSp'] + data['Parch']

11data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)

12

13# 2. Convert Sex column (male=0, female=1)

14data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

15

Average time0.214 s

Maximum time0.214 s

1 out of 1 shown test case(s) passed

Test case 1214 ms

Expected output

Actual output

29.69911764705882

14.4542

3 - 491

1 - 216

2 - 184

Name: Pclass, dtype: int64

TerminalTest cases

Course

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6a/67e3805d7028471d94dae951

202501050060@mitaoe.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarkation location (Embarked).

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

```
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10
11 # 1. Survival rate by class
12 print(data.groupby('Pclass')['Survived'].mean())
13
14 # 2. Survival rate by Embarked_S
15 print(data.groupby('Embarked_S')['Survived'].mean())
```

Average time 0.247 s 247.00 ms Maximum time 0.247 s 247.00 ms 1 out of 1 shown test case(s) passed

Test case 1 247 ms Debug

Expected output	Actual output
Pclass	Pclass
1...0.629630	1...0.629630
2...0.472826	2...0.472826
3...0.242363	3...0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S
0...0.629630	0...0.629630

Terminal Test cases

37°C Partly sunny

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e387fc7028471d94db0238

CODETANTRA Home Learn Anywhere 202501050060@mitaoe.ac.in Support Logout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4 03:45

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9
10 # 1. Survivors by gender
11 print(data[data['Survived'] == 1]['Sex'].value_counts())
12
13 # 2. Non-survivors by gender
14 print(data[data['Survived'] == 0]['Sex'].value_counts())
15
```

Average time 0.214 s 214.00 ms Maximum time 0.214 s 214.00 ms 1 out of 1 shown test case(s) passed

Test case 1 214 ms Debug

Expected output	Actual output
female...233	female...233
male...109	male...109
Name: Sex, dtype: int64	Name: Sex, dtype: int64
female...468	female...468
male...81	male...81
Name: Sex, dtype: int64	Name: Sex, dtype: int64

Terminal Test cases

37°C Partly sunny

Search

17:45 06-05-2026